05. 메시지 집계 시스템

목차

1. 메시지 집계 개념
2. 배치 생성 및 관리
3. 우선순위 메시지 처리
4. 암호화 배치 검증
5. 압축 및 네트워크 최적화
6. 성능 통계 및 모니터링

메시지 집계 개념

`MessageAggregator` 는 네트워크 오버헤드를 줄이고 확장성을 향상시키기 위해 효율적인 메시지 배치 처리와 암호화 배치 검증을 제공하는 핵심 컴포넌트입니다.

기본 구조

```
pub struct MessageAggregator {
config: AggregatorConfig, // 집계 설정
pending_messages: HashMap<Round, Vec<Message>>, // 대기 중인 메시지
pending_batches: VecDeque<MessageBatch>, // 대기 중인 배치
batch_timeouts: HashMap<Round, Instant>, // 배치 타임아웃
next_batch_id: u64, // 다음 배치 ID
```

```
next_message_id: u64, // 다음 메시지 ID
stats: AggregationStats, // 집계 통계
}
```

메시지 구조

```
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct Message {
pub round: Round,
pub data: Vec<u8>,
pub signature: Signature,
pub public_key: PublicKey,
pub timestamp: Timestamp,
pub message_id: u64,
}
```

메시지 배치 구조

```
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct MessageBatch {
pub round: Round,
pub messages: Vec<Message>,
pub batch_id: u64,
```

```
pub created_at: Instant,
pub aggregated_signature: Option<AggregateSignature>,
}
```

집계 설정

```
#[derive(Clone, Debug)]
pub struct AggregatorConfig {
pub batch_size_threshold: usize, // 배치 크기 임계값
pub batch_timeout: Duration, // 배치 타임아웃
pub max_pending_batches: usize, // 최대 대기 배치 수
pub compression_enabled: bool, // 압축 활성화
pub priority_message_types: Vec<MessageType>, // 우선순위 메시지 유형
}
```

배치 생성 및 관리

메시지 집계자 생성

```
impl MessageAggregator {
pub fn new() -> Self {
Self::with_config(AggregatorConfig::default())
}
```

```
pub fn with_config(config: AggregatorConfig) -> Self {
Self {
config,
pending_messages: HashMap::new(),
pending_batches: VecDeque::new(),
batch_timeouts: HashMap::new(),
next_batch_id: 0,
next_message_id: 0,
stats: AggregationStats::default(),
}
}
// 하위 호환성을 위한 기본 매개변수 생성자
pub fn with_params(batch_size_threshold: usize, batch_timeout: Duration) -> Se
let config = AggregatorConfig {
batch_size_threshold,
batch_timeout,
..AggregatorConfig::default()
};
Self::with_config(config)
}
}
```

메시지 추가 및 배치 생성

```
impl MessageAggregator {
pub fn add_message(&mut self, mut message: Message) -> Result<Option<Message
// 고유 메시지 ID 할당
message.message_id = self.next_message_id;
self.next_message_id += 1;
let round = message.round;
// 대기 큐에 메시지 추가
self.pending_messages
.entry(round)
.or_insert_with(Vec::new)
.push(message);
// 이 라운드에 대한 타임아웃이 설정되지 않은 경우 설정
if !self.batch_timeouts.contains_key(&round) {
self.batch_timeouts.insert(round, Instant::now());
}
// 배치 생성 여부 확인
if self.should_create_batch(round) {
self.create_batch(round)
} else {
Ok(None)
}
}
```

```
}
```

배치 생성 조건 확인

```
impl MessageAggregator {
fn should_create_batch(&self, round: Round) -> bool {
if let Some(messages) = self.pending_messages.get(&round) {
// 크기 임계값 도달
if messages.len() >= self.config.batch_size_threshold {
return true;
}
// 타임아웃 임계값 도달
if let Some(start_time) = self.batch_timeouts.get(&round) {
if start_time.elapsed() >= self.config.batch_timeout {
return true;
}
}
}
false
}
}
```

배치 생성 실행

```
impl MessageAggregator {

fn  create_batch(&mut  self, round: Round) -> Result<Option<MessageBatch>, Autob

if  let Some(messages) = self.pending_messages.remove(&round) {

if  messages.is_empty() {

return Ok(None);

}

let  batch_id = self.next_batch_id;

self.next_batch_id += 1;

let  batch = MessageBatch::new(round, messages, batch_id);

// 이 라운드에 대한 타임아웃 추적 제거

self.batch_timeouts.remove(&round);

// 통계 업데이트

self.stats.total_messages_processed += batch.size() as u64;

self.stats.total_batches_created += 1;

self.update_average_batch_size();

// 대기 배치 큐에 추가

self.pending_batches.push_back(batch.clone());

// 메모리 문제 방지를 위해 대기 배치 제한

while  self.pending_batches.len() > self.config.max_pending_batches {

self.pending_batches.pop_front();

}

Ok(Some(batch))
```

```
} else {
Ok(None)
}
}
}
```

타임아웃 처리

```
impl MessageAggregator {
pub fn process_timeouts(&mut self) -> Result<Vec<MessageBatch>, AutobahnError
let mut timed_out_batches = Vec::new();
let mut rounds_to_process = Vec::new();
// 타임아웃된 라운드 찾기
for (&round, &start_time) in &self.batch_timeouts {
if start_time.elapsed() >= self.config.batch_timeout {
rounds_to_process.push(round);
}
}
// 타임아웃된 라운드에 대한 배치 생성
for round in rounds_to_process {
if let Some(batch) = self.create_batch(round)? {
timed_out_batches.push(batch);
}
```

```
}
Ok(timed_out_batches)
}
}
```

우선순위 메시지 처리

메시지 유형 정의

```
#[derive(Clone, Debug, PartialEq, Eq, Hash, Serialize, Deserialize)]
pub enum MessageType {
FastPathVote, // Fast Path 투표
SlowPathVote, // Slow Path 투표
Certificate, // 인증서
Synchronization, // 동기화
Heartbeat, // 하트비트
}
```

우선순위 메시지 추가

```
impl MessageAggregator {
pub fn add_priority_message(&mut self, message: Message, message_type: Messag
// 우선순위 메시지는 중요한 컨센서스 메시지에 대해 즉시 배치 생성을 트리거할 수 있음
```

```
if  self.config.priority_message_types.contains(&message_type) {

let  round = message.round;

let  result = self.add_message(message)?;

// 높은 우선순위 메시지의 경우, 임계값에 도달하지 않아도 배치 생성 고려

if  result.is_none() && matches!(message_type, MessageType::FastPathVote | Messa

if  let Some(messages) = self.pending_messages.get(&round) {

if !messages.is_empty() {

return  self.create_batch(round);

}

}

}

Ok(result)

} else {

self.add_message(message)

}

}

}
```

암호화 배치 검증

배치 검증

```
impl MessageAggregator {
```

```
pub fn verify_batch(&mut self, batch: &mut MessageBatch) -> Result<bool, Auto

if batch.messages.is_empty() {

return Ok(true);

}

// 효율성을 위해 먼저 배치 검증 시도

match self.verify_batch_signatures(&batch.messages) {

Ok(aggregated_sig) => {

batch.aggregated_signature = Some(aggregated_sig);

self.stats.batch_verification_success_rate =

(self.stats.batch_verification_success_rate * (self.stats.total_batches_created

/ self.stats.total_batches_created as f64;

Ok(true)

}

Err(_) => {

// 배치 검증 실패 시 개별 검증으로 폴백

self.verify_batch_individually(batch)

}

}

}

}
```

배치 서명 검증

```
impl MessageAggregator {
fn  verify_batch_signatures(&self, messages: &[Message]) -> Result<AggregateSign
if  messages.is_empty() {
return Err(AutobahnError::AggregationFailed {
reason: "Cannot verify empty message batch".to_string(),
});
}
// 모든 서명과 공개키 수집
let  signatures: Vec<&Signature> = messages.iter().map(|msg| &msg.signature).col
let  public_keys: Vec<&PublicKey> = messages.iter().map(|msg| &msg.public_key).c
// 검증을 위한 메시지 데이터 준비
let  message_data: Vec<&[u8]> = messages.iter().map(|msg| msg.data.as_slice()).c
// 집계 서명 생성
let  aggregated_signature = AggregateSignature::aggregate(&signatures)
.map_err(|e| AutobahnError::AggregationFailed {
reason: format!("Failed to aggregate signatures: {:?}", e),
})?;
// 모든 메시지에 대해 집계된 서명 검증
let  verification_result = aggregated_signature.verify(&message_data, &public_ke
match  verification_result {
Ok(()) => Ok(aggregated_signature),
Err(e) => Err(AutobahnError::AggregationFailed {
```

```
reason: format!("Batch signature verification failed: {:?}", e),
}),
}
}
}
```

개별 메시지 검증 (폴백)

```
impl MessageAggregator {
fn verify_batch_individually(&mut self, batch: &MessageBatch) -> Result<bool,
let mut valid_count = 0;
let mut invalid_messages = Vec::new();
for (index, message) in batch.messages.iter().enumerate() {
match self.verify_single_message(message) {
Ok(true) => valid_count += 1,
Ok(false) => invalid_messages.push(index),
Err(e) => {
return Err(AutobahnError::AggregationFailed {
reason: format!("Individual verification failed for message {}: {:?}", index, e)
});
}
}
}
```

```
// 개별 검증 결과를 기반으로 성공률 업데이트
let  success_rate = valid_count  as f64 / batch.messages.len() as f64;
self.stats.batch_verification_success_rate =
(self.stats.batch_verification_success_rate * (self.stats.total_batches_created
/ self.stats.total_batches_created as f64;
// 메시지의 과반수가 유효한 경우 배치를 유효한 것으로 간주
let  threshold = (batch.messages.len() * 2) / 3; // 2/3 임계값
Ok(valid_count >= threshold)
}
fn  verify_single_message(&self, message: &Message) -> Result<bool, AutobahnErro
use crypto::traits::VerifyingKey;
// 메시지 데이터에 대해 서명 검증
match  message.public_key.verify(&message.data, &message.signature) {
Ok(()) => Ok(true),
Err(e) => {
// 검증 실패를 로그하지만 전체 배치를 실패시키지는 않음
eprintln!("Single message verification failed: {:?}", e);
Ok(false)
}
}
}
}
```

집계된 증명 생성

```
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct AggregatedProof {
pub batch_id: u64,
pub round: Round,
pub aggregated_signature: AggregateSignature,
pub public_keys: Vec<PublicKey>,
pub message_hashes: Vec<Vec<u8>>,
pub participant_count: usize,
}

impl MessageAggregator {
pub fn create_aggregated_proof(&self, batch: &MessageBatch) -> Result<Aggregat
if let Some(ref aggregated_sig) = batch.aggregated_signature {
let public_keys: Vec<PublicKey> = batch.messages.iter().map(|msg| msg.public_ke
let message_hashes: Vec<Vec<u8>> = batch.messages.iter()
.map(|msg| self.hash_message_data(&msg.data))
.collect();
Ok(AggregatedProof {
batch_id: batch.batch_id,
round: batch.round,
aggregated_signature: aggregated_sig.clone(),
```

```
public_keys,
message_hashes,
participant_count: batch.messages.len(),
})
} else {
Err(AutobahnError::AggregationFailed {
reason: "Batch has no aggregated signature".to_string(),
})
}
}
}
```

압축 및 네트워크 최적화

압축된 배치 구조

```
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct CompressedBatch {
pub batch_id: u64,
pub round: Round,
pub compressed_data: Vec<u8>,
pub compression_ratio: f64,
pub original_size: usize,
```

```
}
```

배치 압축

```
impl MessageAggregator {
pub fn compress_batch(&self, batch: &MessageBatch) -> Result<CompressedBatch,
if !self.config.compression_enabled {
return Ok(CompressedBatch {
batch_id: batch.batch_id,
round: batch.round,
compressed_data: self.serialize_batch(batch)?,
compression_ratio: 1.0,
original_size: batch.total_data_size(),
});
}
let original_data = self.serialize_batch(batch)?;
let original_size = original_data.len();
// 간단한 압축 (실제로는 zstd나 lz4 같은 적절한 압축 라이브러리 사용)
let compressed_data = self.simple_compress(&original_data);
let compressed_size = compressed_data.len();
let compression_ratio = if original_size > 0 {
compressed_size as f64 / original_size as f64
} else {
```

```
1.0
};
Ok(CompressedBatch {
batch_id: batch.batch_id,
round: batch.round,
compressed_data,
compression_ratio,
original_size,
})
}
}
```

간단한 압축 구현 (RLE)

```
impl MessageAggregator {
fn simple_compress(&self, data: &[u8]) -> Vec<u8> {
// 데모용 매우 기본적인 런-렝스 인코딩
// 프로덕션에서는 적절한 압축 라이브러리 사용
let mut compressed = Vec::new();
if data.is_empty() {
return compressed;
}
let mut current_byte = data[0];
```

```
let mut count = 1u8;
for &byte in &data[1..] {
if byte == current_byte && count < 255 {
count += 1;
} else {
compressed.push(count);
compressed.push(current_byte);
current_byte = byte;
count = 1;
}
}
compressed.push(count);
compressed.push(current_byte);
compressed
}
fn simple_decompress(&self, compressed: &[u8]) -> Result<Vec<u8>, AutobahnError
let mut decompressed = Vec::new();
if compressed.len() % 2 != 0 {
return Err(AutobahnError::AggregationFailed {
reason: "Invalid compressed data format".to_string(),
});
}
for chunk in compressed.chunks(2) {
```

```
let count = chunk[0];
let byte = chunk[1];
for _ in 0..count {
decompressed.push(byte);
}
}
Ok(decompressed)
}
}
```

성능 통계 및 모니터링

집계 통계 구조

```
#[derive(Clone, Debug, Default)]
pub struct AggregationStats {
pub total_messages_processed: u64,
pub total_batches_created: u64,
pub average_batch_size: f64,
pub batch_verification_success_rate: f64,
pub compression_ratio: f64,
pub network_bytes_saved: u64,
```

```
}
```

통계 업데이트 및 조회

```
impl MessageAggregator {
fn update_average_batch_size(&mut self) {
if self.stats.total_batches_created > 0 {
self.stats.average_batch_size =
self.stats.total_messages_processed as f64 / self.stats.total_batches_created as
}
}
pub fn get_stats(&self) -> &AggregationStats {
&self.stats
}
pub fn reset_stats(&mut self) {
self.stats = AggregationStats::default();
}
}
```

배치 관리 유틸리티

```
impl MessageAggregator {
pub fn get_next_batch(&mut self) -> Option<MessageBatch> {
```

```
self.pending_batches.pop_front()
}
pub fn get_all_pending_batches(&mut self) -> Vec<MessageBatch> {
self.pending_batches.drain(..).collect()
}
pub fn pending_count(&self, round: Round) -> usize {
self.pending_messages
.get(&round)
.map(|messages| messages.len())
.unwrap_or(0)
}
pub fn total_pending_count(&self) -> usize {
self.pending_messages.values().map(|msgs| msgs.len()).sum()
}
pub fn pending_batch_count(&self) -> usize {
self.pending_batches.len()
}
}
```

메모리 관리 및 정리

```
impl MessageAggregator {
pub fn cleanup_old_messages(&mut self, keep_rounds: usize, current_round: Rou
```

```
let cutoff_round = if current_round.0 > keep_rounds as u64 {
Round(current_round.0 - keep_rounds as u64)
} else {
Round(0)
};
// 오래된 대기 메시지 정리
self.pending_messages.retain(|&round, _| round >= cutoff_round);
// 배치 타임아웃 정리
self.batch_timeouts.retain(|&round, _| round >= cutoff_round);
// 오래된 배치 정리 (5분)
let cutoff_time = Instant::now() - Duration::from_secs(300);
self.pending_batches.retain(|batch| batch.created_at > cutoff_time);
}
}
```

메시지 집계 최적화 전략

네트워크 조건별 설정

고처리량 환경

```
let config = AggregatorConfig {
batch_size_threshold: 200,
batch_timeout: Duration::from_millis(50),
```

```
max_pending_batches: 1000,
compression_enabled: true,
priority_message_types: vec![MessageType::FastPathVote, MessageType::Certificate
};
```

저지연 환경

```
let  config = AggregatorConfig {
batch_size_threshold: 25,
batch_timeout: Duration::from_millis(10),
max_pending_batches: 100,
compression_enabled: false,
priority_message_types: vec![MessageType::FastPathVote, MessageType::Certificate
};
```

동적 배치 크기 조정

```
impl MessageAggregator {
pub  fn  adaptive_batching_example(&mut  self) {
let  batch_stats = self.get_stats();
if  batch_stats.average_batch_size < 50.0 {
// 타임아웃을 늘려서 더 큰 배치 허용
self.config.batch_timeout = Duration::from_millis(100);
```

```
} else if batch_stats.average_batch_size > 200.0 {

// 타임아웃을 줄여서 지연시간 감소

self.config.batch_timeout = Duration::from_millis(25);

}

}

}
```

메시지 집계 시스템은 네트워크 효율성을 크게 향상시키며, 암호화 배치 검증을 통해 보안을 유지하면서 처리량을 최적화합니다.